

# ASSIGNMENT 3C WRITE-UP

## Emotion Classifier + LLM Critic

**Notebook:** assignment3c\_complete\_pipeline.ipynb

**Dataset:** Hugging Face: dair-ai/emotion

**Model:** TF-IDF + Logistic Regression

**Tooling route:** Google Colab + VS Code + MCP + Groq or xAI/Grok API

**Results status:** Fill in accuracy and weakest-class F1 after running Section 3 of the notebook

---

### What I built

I built a small end-to-end AI + ML pipeline around emotion classification. The machine-learning component is a TF-IDF + Logistic Regression classifier trained on the `dair-ai/emotion` dataset, which contains short English text examples labeled with six emotions: sadness, joy, love, anger, fear, and surprise.

After training, the notebook evaluates the classifier with a classification report and confusion matrix, uses LIME as an independent local explainability check, and saves the trained model as `emotion\_model.joblib` with `label\_names.txt`.

The AI-in-the-loop component is handled in two ways: the notebook can send the model output to Groq or xAI/Grok as a critic, and it also generates an `mcp\_server.py` file that exposes the classifier through a `predict\_emotion` MCP tool. I used VS Code as the local environment for inspecting artifacts and testing the MCP server after the Colab run.

- **Repo/notebook:** [add your Colab or GitHub link]
- **Test accuracy: 0.8660**
- **Weakest class by F1:** Surprise (0.6261)

### Where the AI tooling genuinely accelerated me

- It sped up the fiddly parts of the implementation, especially extracting TF-IDF feature weights from a sparse matrix so that each prediction could include top contributing words or n-grams.
- It scaffolded the MCP server boilerplate, including the `predict\_emotion` tool, stdio transport, model loading, and the JSON-style response that an LLM can inspect.

- It helped adapt the workflow to the tools I had: Google Colab for training, VS Code for local testing, and Groq or xAI/Grok for the LLM critic instead of Claude Code.

## Where it confidently misled me

The most important risk is that an LLM critic can sound more certain than the underlying evidence deserves. For example, on a sarcastic sentence such as `well that was just great, exactly what I needed today`, the classifier may over-weight the word `great` and predict a positive emotion. An LLM may catch the sarcasm, but it may also rationalize the model's label instead of challenging it.

Another risk is explanation mismatch. If the LLM explains the model's behavior using its own interpretation rather than the actual top-weighted TF-IDF terms returned by the tool, the explanation becomes fluent but misleading.

A third risk is tooling opacity. If an MCP call or API call fails, the assistant may still produce a polished answer unless the notebook explicitly prints the tool payload and checks whether the response came from the model, the LLM, or a fallback prompt.

## What would a production version need?

- **Data quality:** `dair-ai/emotion` is a convenient benchmark dataset, not a production labeling program. A real system would need documented labeling rules, inter-annotator agreement, domain-specific examples, language coverage, and checks for class imbalance or annotation bias.
- **Evaluation beyond accuracy:** The notebook already reports per-class precision, recall, and F1, but production would also need calibration, threshold analysis, robustness tests for negation and sarcasm, and held-out data from the actual target domain.
- **LLM critic reliability:** The critic needs its own evaluation: how often it identifies model errors, rubber-stamps weak predictions, or introduces a new mistake.
- **Monitoring:** A deployed version would track model confidence, disagreement between the classifier and LLM critic, latency, API cost, failure rates, and data drift.
- **Safety and process:** Cases where the classifier and LLM disagree should be routed to human review rather than being automatically resolved.

## Tools tried

Google Colab, VS Code, Hugging Face datasets, scikit-learn, LIME, a custom MCP server/MCP Inspector, and Groq or xAI/Grok API for the LLM critic.

## How I would run and test it

- Upload `assignment3c_complete_pipeline.ipynb` to Google Colab and run all cells.
- Add `GROQ_API_KEY` or `XAI_API_KEY` in Colab Secrets if using the optional LLM critic cell.
- Download `assignment3c_artifacts.zip`, extract it, and open the folder in VS Code.
- Create a virtual environment in the VS Code terminal and install `mcp`, `joblib`, `scikit-learn`, and `numpy`.
- Run `mcp dev mcp_server.py` to test the `predict_emotion` MCP tool locally.